Sort can also take a block. The block takes two elements and returns:

- A positive integer if the first argument is, for the purposes of the sort, greater than the second argument.
- A negative integer if the first argument is less than the second argument.
- Zero, if the two elements are tied.

Assuming the objects have a height element, both of these lines will return identical sorts:

```
list_of_users.sort { |a, b| a.height <=> b.height }
list_of_users.sort { |a, b| a.height - b.height }
```

The first line takes advantage of the <=> operator's behavior, matching the expected behavior of the block, while the second line takes advantage of just plain subtraction having the same result. (We'd consider the first line to be clearer in most contexts.)

Sorting elements in an array based on an attribute of each element is common enough that Ruby provides sort_by as a shortcut. The sort_by method takes a block with a single argument and uses that argument to sort the list.

Given all the ways in Ruby to write a short block, the following are all equivalent:

```
list_of_users.sort { |a, b| a.height <=> b.height }
list_of_users.sort_by { |x| x.height }
list_of_users.sort_by { _1.height }
list_of_users.sort_by(&:height)
```

You'll likely see the last form most frequently for simple cases like this, though we do kind-of hope the next-to-last form catches on.

If you are just interested in the upper or lower most elements rather than the entire sorted list, Ruby has you covered with min and max and their related methods min_by and max_by.

When called without an argument, these methods treat the array exactly like sort and sort_by, which is to say that min and max use <=> or take a two argument block, while min_by and max_by take a one argument block. The difference is the return value—the min methods return the lowest element of the list; the max elements return the highest element of the list.

If you want more than one element returned, you can pass the number of elements you want to the method and you will receive an array of results rather than a single scalar result. The method minmax returns a two-element array with the lowest and highest elements of the list using the same logic as sort. The related minmax_by uses the same logic as sort_by. For example:

```
[4, 11, 2, 7]        # => [4, 11, 2, 7]
[4, 11, 2, 7].min    # => 2
[4, 11, 2, 7].max    # => 11
[4, 11, 2, 7].min(2) # => [2, 4]
[4, 11, 2, 7].max(2) # => [11, 7]
[4, 11, 2, 7].minmax # => [2, 11]

["one", "three", "eleven", "four"].min_by(&:length)    # => "one"
["one", "three", "eleven", "four"].max_by(&:length)    # => "eleven"
["one", "three", "eleven", "four"].min_by(2, &:length) # => ["one", "four"]
["one", "three", "eleven", "four"].max_by(2, &:length) # => ["eleven", "three"]
["one", "three", "eleven", "four"].minmax_by(&:length) # => ["one", "eleven"]
```